API Documentation

Overview

The API is a RESTful API that provides endpoints for querying clients, projects, tasks, and managing time entries. The API supports API Key (Bearer Token) based authentication.

Base URL: `https://interview-api.memtime-demo.deno.net/api/v1`

Authentication

Bearer Token (API Key)

All requests to `/api/v1` endpoints require a Bearer token in the `Authorization` header.

```
Authorization: Bearer <API_KEY>
```

Rate Limiting

- **Window:** 60 seconds (1 minute)
- **Limit:** 15 requests per window
- Rate limit key is based on the API key provided

Response Format

All responses are returned as JSON with pretty-printing enabled.

Data Types

Client

```
{
  "id": 1,
  "name": "Client Name",
  "description": "Client description",
  "status": "active",
  "createdAt": "2024-01-15T10:30:00.000Z",
```

```
  "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

Project

```
{
  "id": 1,
  "clientId": 1,
  "name": "Project Name",
  "status": "active",
  "createdAt": "2024-01-15T10:30:00.000Z",
  "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

Task

```
{
  "id": 1,
  "parent": 0,
  "name": "Task Name",
  "status": "active",
  "createdAt": "2024-01-15T10:30:00.000Z",
  "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

TimeEntry

```
{
  "id": 1,
  "taskId": 1,
  "userId": "<API_KEY>",
  "comment": "Worked on feature development",
  "start": "2024-01-15T09:00:00.000Z",
  "end": "2024-01-15T17:00:00.000Z",
  "createdAt": "2024-01-15T17:05:00.000Z",
  "updatedAt": "2024-01-15T17:05:00.000Z"
}
```

Pagination

List endpoints support pagination with the following query parameters:

Parameter	Type	Default	Description
`limit`	number	10	Maximum number of results to return
`offset`	number	0	Number of results to skip

Parameter	Type	Default	Description
`sortBy`	string	-	Field name to sort by (must be a valid field)
`order`	string	asc	Sort order: `asc` or `desc`

Example:

```
GET /api/v1/clients?limit=20&offset=0&sortBy=name&order=asc
```

Endpoints

Clients

Get All Clients

Retrieve a paginated list of all clients.

```
GET /api/v1/clients
```

Query Parameters: - `limit` (optional): Number of results per page (default: 10) - `offset` (optional): Number of results to skip (default: 0) - `sortBy` (optional): Field to sort by - `order` (optional): Sort order `asc` or `desc` (default: `asc`)

Response: `200 OK`

```
[
  {
    "id": 1,
    "name": "Acme Corp",
    "description": "Leading supplier",
    "status": "active",
    "createdAt": "2024-01-15T10:30:00.000Z",
    "updatedAt": "2024-01-15T10:30:00.000Z"
  }
]
```

Example:

```
curl -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/clients?limit=10&offset=0
```

Get Client by ID

Retrieve a specific client by ID.

```
GET /api/v1/clients/:id
```

Path Parameters: - `id` (required): Client ID

Response: `200 OK`

```
{
  "id": 1,
  "name": "Acme Corp",
  "description": "Leading supplier",
  "status": "active",
  "createdAt": "2024-01-15T10:30:00.000Z",
  "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

Error Responses: - `404 Not Found` - Client not found

```
{
  "error": "Client not found"
}
```

Example:

```
curl -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/clients/1
```

Projects

Get Projects of a Client

Retrieve all projects belonging to a specific client.

```
GET /api/v1/clients/:id/projects
```

Path Parameters: - `id` (required): Client ID

Query Parameters: - `limit` (optional): Number of results per page (default: 10) - `offset` (optional): Number of results to skip (default: 0) - `sortBy` (optional): Field to sort by - `order` (optional): Sort order `asc` or `desc` (default: `asc`)

Response: `200 OK`

```
[
  {
    "id": 1,
    "clientId": 1,
    "name": "Website Redesign",
    "status": "active",
    "createdAt": "2024-01-15T10:30:00.000Z",
    "updatedAt": "2024-01-15T10:30:00.000Z"
  }
]
```

Example:

```
curl -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/clients/1/projects
```

Get Project by ID

Retrieve a specific project by ID.

`GET /api/v1/projects/:id`

Path Parameters: - `id` (required): Project ID

Response: `200 OK`

```
{
  "id": 1,
  "clientId": 1,
  "name": "Website Redesign",
  "status": "active",
  "createdAt": "2024-01-15T10:30:00.000Z",
  "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

Error Responses: - `404 Not Found` - Project not found

```
{
  "error": "Project not found"
}
```

Example:

```
curl -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/projects/1
```

Tasks

Get Tasks of a Project

Retrieve all tasks belonging to a specific project.

`GET /api/v1/projects/:id/tasks`

Path Parameters: - `id` (required): Project ID

Query Parameters: - `limit` (optional): Number of results per page (default: 10) - `offset` (optional): Number of results to skip (default: 0) - `sortBy` (optional): Field to sort by - `order` (optional): Sort order `asc` or `desc` (default: `asc`)

Response: `200 OK`

```
[
  {
```

```
    "id": 1,
    "parent": 0,
    "name": "Design mockups",
    "status": "active",
    "createdAt": "2024-01-15T10:30:00.000Z",
    "updatedAt": "2024-01-15T10:30:00.000Z"
  }
]
```

Example:

```
curl -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/projects/1/tasks
```

Get Task by ID

Retrieve a specific task by ID.

`GET /api/v1/tasks/:id`

Path Parameters: - `id` (required): Task ID

Response: `200 OK`

```
{
  "id": 1,
  "parent": 0,
  "name": "Design mockups",
  "status": "active",
  "createdAt": "2024-01-15T10:30:00.000Z",
  "updatedAt": "2024-01-15T10:30:00.000Z"
}
```

Error Responses: - `404 Not Found` - Task not found

```
{
  "error": "Task not found"
}
```

Example:

```
curl -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/tasks/1
```

Time Entries

Get My Time Entries

Retrieve all time entries for the authenticated user.

`GET /api/v1/time-entries`

Query Parameters: - `limit` (optional): Number of results per page (default: 10) - `offset` (optional): Number of results to skip (default: 0) - `sortBy` (optional): Field to sort by - `order` (optional): Sort order `asc` or `desc` (default: `asc`)

Response: `200 OK`

```
[
  {
    "id": 1,
    "taskId": 1,
    "userId": "<API_KEY>",
    "comment": "Worked on feature development",
    "start": "2024-01-15T09:00:00.000Z",
    "end": "2024-01-15T17:00:00.000Z",
    "createdAt": "2024-01-15T17:05:00.000Z",
    "updatedAt": "2024-01-15T17:05:00.000Z"
  }
]
```

Error Responses: - `404 Not Found` - User not found/authenticated

```
{
  "error": "Unknown user"
}
```

Example:

```
curl -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/time-entries
```

Get Time Entry by ID

Retrieve a specific time entry by ID. Only returns entries belonging to the authenticated user.

```
GET /api/v1/time-entries/:id
```

Path Parameters: - `id` (required): Time Entry ID

Response: `200 OK`

```
{
  "id": 1,
  "taskId": 1,
  "userId": "<API_KEY>",
  "comment": "Worked on feature development",
  "start": "2024-01-15T09:00:00.000Z",
  "end": "2024-01-15T17:00:00.000Z",
  "createdAt": "2024-01-15T17:05:00.000Z",
  "updatedAt": "2024-01-15T17:05:00.000Z"
}
```

Error Responses: - `404 Not Found` - Time entry not found or does not belong to user

```
{
  "error": "Time entry not found"
}
```

Example:

```
curl -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/time-entries/1
```

Create Time Entry

Create a new time entry for the authenticated user.

```
POST /api/v1/time-entries
```

Request Body:

```
{
  "taskId": 1,
  "comment": "Worked on feature development",
  "start": "2024-01-15T09:00:00Z",
  "end": "2024-01-15T17:00:00Z"
}
```

Body Parameters: - `taskId` (required): Task ID associated with the time entry - `comment` (required): Comment/description for the time entry - `start` (required): Start time (ISO 8601 format) - `end` (required): End time (ISO 8601 format)

Response: `201 Created`

```
{
  "id": 2,
  "taskId": 1,
  "userId": "<API_KEY>",
  "comment": "Worked on feature development",
  "start": "2024-01-15T09:00:00.000Z",
  "end": "2024-01-15T17:00:00.000Z",
  "createdAt": "2024-01-15T17:15:00.000Z",
  "updatedAt": "2024-01-15T17:15:00.000Z"
}
```

Error Responses: - `400 Bad Request` - Invalid JSON body or missing required fields

```
{
  "error": "Invalid JSON body"
}
```

Example:

```
curl -X POST \
  -H "Authorization: Bearer <API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{
```

```
    "taskId": 1,
    "comment": "Worked on feature development",
    "start": "2024-01-15T09:00:00Z",
    "end": "2024-01-15T17:00:00Z"
  }' \
  https://api.example.com/api/v1/time-entries
```

Update Time Entry

Update an existing time entry.

`PUT /api/v1/time-entries/:id`

Path Parameters: - `id` (required): Time Entry ID

Request Body:

```
{
  "taskId": 1,
  "comment": "Updated comment",
  "start": "2024-01-15T09:30:00Z",
  "end": "2024-01-15T17:30:00Z"
}
```

Body Parameters: - `taskId` (required): Task ID associated with the time entry - `comment` (required): Comment/description for the time entry - `start` (required): Start time (ISO 8601 format) - `end` (required): End time (ISO 8601 format)

Response: `200 OK`

```
{
  "id": 1,
  "taskId": 1,
  "userId": "<API_KEY>",
  "comment": "Updated comment",
  "start": "2024-01-15T09:30:00.000Z",
  "end": "2024-01-15T17:30:00.000Z",
  "createdAt": "2024-01-15T17:05:00.000Z",
  "updatedAt": "2024-01-15T17:20:00.000Z"
}
```

Error Responses: - `400 Bad Request` - Time entry ID is required for update

```
{
  "error": "Time entry ID is required for update"
}
```

- `404 Not Found` - Time entry not found or does not belong to user

```
{
  "error": "Time entry #1 not found"
}
```

- `400 Bad Request` - Invalid JSON body

```
{
  "error": "Invalid JSON body"
}
```

Example:

```
curl -X PUT \
  -H "Authorization: Bearer <API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{
    "taskId": 1,
    "comment": "Updated comment",
    "start": "2024-01-15T09:30:00Z",
    "end": "2024-01-15T17:30:00Z"
  }' \
  https://api.example.com/api/v1/time-entries/1
```

Delete Time Entry

Delete an existing time entry.

```
DELETE /api/v1/time-entries/:id
```

Path Parameters: - `id` (required): Time Entry ID

Response: `200 OK`

```
{
  "message": "Time entry deleted successfully"
}
```

Error Responses: - `404 Not Found` - Time entry not found or does not belong to user

```
{
  "error": "Time entry not found"
}
```

Example:

```
curl -X DELETE \
  -H "Authorization: Bearer <API_KEY>" \
  https://api.example.com/api/v1/time-entries/1
```

Error Handling

The API returns standard HTTP status codes along with JSON error messages:

Status Code	Description
`200`	OK - Request successful
`201`	Created - Resource created successfully
`400`	Bad Request - Invalid request parameters or body
`404`	Not Found - Resource not found
`429`	Too Many Requests - Rate limit exceeded
`500`	Internal Server Error - Server error

Error Response Format:

```
{
  "error": "Error description"
}
```

Rate Limiting

The API implements rate limiting to prevent abuse:

- **Window:** 60 seconds
- **Limit:** 15 requests per window
- **Key:** Based on the API key used

When the rate limit is exceeded, the API returns a `429 Too Many Requests` status code.

Request Headers

All requests must include:

```
Authorization: Bearer <API_KEY>
```

Optional headers:

```
Content-Type: application/json  (for POST/PUT requests)
```

Summary of Endpoints

Method	Endpoint	Description
`GET`	`/clients`	Get all clients
`GET`	`/clients/:id`	Get a specific client
`GET`	`/clients/:id/projects`	Get projects for a client
`GET`	`/projects/:id`	Get a specific project
`GET`	`/projects/:id/tasks`	Get tasks for a project

Method	Endpoint	Description
GET	/tasks/:id	Get a specific task
GET	/time-entries	Get user's time entries
GET	/time-entries/:id	Get a specific time entry
POST	/time-entries	Create a new time entry
PUT	/time-entries/:id	Update a time entry
DELETE	/time-entries/:id	Delete a time entry

Health Check

To verify the API is running:

```
GET /health
```

Response: `200 OK`

```
I'm alive!
```